Freestanding Library: Partial Classes

Document number: P2407R3
Date: 2023-03-05
Reply-to:

Ben Craig <ben dot craig at gmail dot com>
Emil Meissner <e dot meissner at seznam dot cz>

Audience: Library Working Group

Changes from previous revisions

Changes from R2

- `// freestanding-delete` to *`// freestanding-deleted`*
- Adding `bad_optional_access` per LWG's request for consistency
- Rewording `string_view::starts_with` and `string_view::ends_with` in terms of freestanding functions
- Avoiding word "item" in instructions to the editor
- Made example in wording use *`// freestanding-deleted`*
- Added *`// mostly freestanding`* header marker

Changes from R1

- Ported paper to HTML
- Only annotating the deleted parts of partial classes, and not the included parts
- Mentioning forward_like in relation to variant's value categories
- New prose for [freestanding.item]

Changes from R0

- Add wording for feature test macros
- Mention monadic optional and `string_view::contains`

Introduction

This proposal is part of a group of papers aimed at improving the state of freestanding. It marks (parts of) std::array, std::string_view, std::variant, and std::optional as such. A future paper might add std::bitset (as was the original goal in [P2268R0])

Motivation and Scope

All of the added classes are fundamentally compatible with freestanding, except for a few methods that throw (e.g. array::at). We explicitly =delete these undesirable methods.

The main driving factor for these additions is the immense usefulness of these types in practice.

Scope

We refine [freestanding.item] by specifying the notion of partial classes, and accordingly specify the newly (partially) freestanding classes as such.

About <bitset>

As mentioned in the introduction, this paper does not deal with bitset. Bitset is unique in that a relatively big part of its interface depends on std::basic_string. We do not currently have a sound plan to make bitset work as nicely as we'd like to. This situation is made worse by a significant amount of bitset's member functions that throw.

Implementation experience

The Existing Standard Library

We've forked libc++, and =deleted all not freestanding methods. Except for some methods on string_view (which are implemented in terms of the deleted string_view::substring), this did not require any changes in the implementation. All test cases (except for the deleted methods) passed after some rather minor adjustments (e.g. replacing get<0>(v) with *get_if<0>(&v)), confirming that all these types are usable without the deleted methods.

In Practice

Since we aren't changing the semantics of any of the classes (except deleted non-critical methods), it is fair to say that all of the (implementer and user) experience gathered as part of hosted applies the same to freestanding.

The only question is, whether these classes are compatible with freestanding. To which the answer is yes! For example, the [Embedded Template Library] offers direct mappings of the std types. Even in kernel-level libraries, like Serenity's [AK] use a form of these utilities.

Design decisions

Deleting behavior

Our decision to delete methods we can't mark as freestanding was made to keep overload resolution the same on freestanding as hosted.

An additional benefit here is, that users of these classes, who might expect to use a throwing method, which was not provided by the implementation, will get a more meaningful error than the method simply missing. This also means we can keep options open for reintroducing the deleted functions into freestanding. (e.g. operator<<(ostream, string_view), should <ostream> be added).

[conventions] changes

The predecessor to this paper used `//freestanding, partial` to mean a class (template) is only required to be partially implemented, in conjunction with `//freestanding, omit` meaning a declaration is not in freestanding.

In this paper, we mark not fully freestanding classes templates as `// freestanding-partial`, and use P2338's `// freestanding-deleted` to mark which pieces of the class should be omitted. We no longer annotate all the class members, favoring terseness over explicitness.

On std::visit

In this paper, we mark std::visit as freestanding, even though it is theoretically throwing. However, the conditions for std::visit to throw are as follows:

It is possible for a variant to hold no value if an exception is thrown during a type-changing assignment or emplacement.

This means a variant will only throw on visit if a user type throws (library types don't throw on freestanding). In this case, std::visit throwing isn't a problem, since the user's code is already using, and (hopefully) handling exceptions.

This however has the unfortunate side-effect that we need to keep bad_variant_access freestanding.

Notes on variant and value categories

By getting rid of std::get, we force users to use std::get_if. Since std::get_if returns a pointer, one can only access the value of a variant by dereferencing said pointer, obtaining an lvalue, discarding the value category of the held object. This is unlikely to have an impact on application code, but might impact highly generic library code.

std::forward_like can help in these cases. The value category of the variant can be transferred to the dereferenced pointer returned from set::get_if.

Justification for deletions

Every deleted method is throwing. We omit string_view's associated operator<< since we don't add basic_ostream.

Monadic optional and string_view::contains

Since this paper was first published, std::string_view got a new contains member function, and std::optional got transform, and_then, and or_else. All these functions are not throwing, and there are no other problems regarding freestanding. We therefore opt for them being marked as freestanding.

Wording

This paper's wording is based on the current working draft, [N4928], and it assumes that [P2338R3] has been applied.

Change in [freestanding.item]

Modify [freestanding.item].

3 A declaration in a header synopsis is a freestanding item if

(3.1) • it is followed by a comment that includes *freestanding*, or

(3.2) • the header synopsis begins with a comment that includes *all freestanding*~~.~~**, or**

(3.3) • **the header synopsis begins with a comment that includes *mostly freestanding* and the declaration is not followed by a comment that includes**
  - ***freestanding-partial*, or**
  - ***freestanding-deleted*, or**

- ***hosted*.**

...

5 A macro is a freestanding item if it is defined in a header synopsis and

- (5.1) the definition is followed by a comment that includes *freestanding*, or
- (5.2) the header synopsis begins with a comment that includes *all freestanding*~~.~~**, or**
- **(5.3) the header synopsis begins with a comment that includes *mostly freestanding* and the definition is not followed by a comment that includes**
  - ***freestanding-partial*, or**
  - ***freestanding-deleted*, or**
  - ***hosted*.**

Drafting note: Apply the following change after applying the changes in P2338 "Freestanding Library: Character primitives and the C library".
Add a new paragraph to [freestanding.item].

? A class type declaration or class template declaration in a header synopsis that is followed by a comment that includes *freestanding-partial* is a freestanding item, except that it contains at least one freestanding deleted function.
[*Example:*

```
template <class T, size_t N> struct array; //freestanding-partial

template<class T, size_t N>
struct array {
  constexpr reference       operator[](size_type n);
  constexpr const_reference operator[](size_type n) const;
  constexpr reference       at(size_type n); //freestanding-deleted
  constexpr const_reference at(size_type n) const; //freestanding-deleted
};
```

-end example]

Changes in [compliance]

Add new rows to the "C++ headers for freestanding implementations" table:

Subclause		Header(s)
[...]	[...]	[...]
?.? [optional]	**Optional objects**	**`<optional>`**
?.? [variant]	**Variants**	**`<variant>`**
?.? [string.view]	**String view classes**	**`<string_view>`**
?.? [array]	**Class template `array`**	**`<array>`**
[...]	[...]	[...]

Changes in [optional.syn]

Instructions to the editor:

Please insert a `// mostly freestanding` comment at the beginning of the [optional.syn] synopsis.

Please append a `// freestanding-partial` comment to the following declaration:

- `optional`

Changes in [optional.optional.general]

Instructions to the editor:

Please append a `// freestanding-deleted` comment to every overload of `value`.

Changes in [variant.syn]

Instructions to the editor:

Please insert a `// mostly freestanding` comment at the beginning of the [variant.syn] synopsis.

Please append a `// freestanding-deleted` comment to every `get` overload in the synopsis.

Changes in [string.view.synop]

Instructions to the editor:

Please insert a `// mostly freestanding` comment at the beginning of the [string.view.synop] synopsis.

Please append a `// hosted` comment to the following declaration:

- `operator<<`

Please append a `// freestanding-partial` comment to the following declaration:

- `basic_string_view`

Changes in [string.view.template.general]

Instructions to the editor:
Please append a `// freestanding-deleted` to the following functions:

- `at`
- `copy`
- `substr`
- The following overloads of compare:
    - `compare(size_type pos1, size_type n1, basic_string_view s)`
    - `compare(size_type pos1, size_type n1, basic_string_view s, size_type pos2, size_type n2)`
    - `compare(size_type pos1, size_type n1, const charT* s)`
    - `compare(size_type pos1, size_type n1, const charT* s, size_type n2)`

Note that the `compare(basic_string_view str) const` and `compare(const charT* s) const` overloads are intentionally not freestanding-deleted.

Changes in [string.view.ops]

Please modify the `basic_string_view` overload of `starts_with` so that it doesn't reference *freestanding-deleted* methods.

```
constexpr bool starts_with(basic_string_view x) const noexcept;
```

? **Let `rlen` be the smaller of `size()` and `x.size()`.**

? *Effects*: Equivalent to: ~~`return substr(0, x.size()) == x;`~~ **`return basic_string_view(data(), rlen) == x;`**

Please modify the `basic_string_view` overload of `ends_with` so that it doesn't reference *freestanding-deleted* methods.

```
constexpr bool ends_with(basic_string_view x) const noexcept;
```

? **Let `rlen` be the smaller of `size()` and `x.size()`.**

? *Effects*: Equivalent to: ~~`return size() >= x.size() && compare(size() - x.size(), npos, x) == 0`~~ **`return basic_string_view(data() + (size - rlen), rlen) == x;`**

Changes in [array.syn]

Instructions to the editor:

Please insert a *`// mostly freestanding`* comment at the beginning of the [array.syn] synopsis.

Please append a *`// freestanding-partial`* comment to `array`

Changes in [array.overview]

Instructions to the editor:
Please append a *`// freestanding-deleted`* comment to every overload of `at`.

Changes in [version.syn]

This part of the paper follows the guide lines as specified in [P2198R6]. Instructions to the editor:
Add the following macros to [version.syn]:

```
#define __cpp_lib_freestanding_array 20XXXXL //also in <array>
#define __cpp_lib_freestanding_optional 20XXXXL //also in <optional>
#define __cpp_lib_freestanding_string_view 20XXXXL //also in <string_view>
#define __cpp_lib_freestanding_variant 20XXXXL //also in <variant>
```

References

[AK] Andreas Kling. Serenity OS AK Library.
https://github.com/SerenityOS/serenity/tree/master/AK
[Embedded Template Library] John Wellbelove. Embedded Template Library.
https://www.etlcpp.com/
[N4928] Thomas Köppe. 2022-12-18. Working Draft, Standard for Programming Language C++.

https://wg21.link/n4928
[P2198R6] Ben Craig. 2022-12-06. Freestanding Feature-Test Macros and Implementation-Defined Extensions.
https://wg21.link/P2198R6
[P2268R0] Ben Craig. 2020-12-10. Freestanding Roadmap.
https://wg21.link/p2268r0
[P2338R3] Ben Craig. 2022-12-06. Freestanding Library: Character primitives and the C library.
https://wg21.link/P2338R3